LONGEST INCREASING SUBSEQUENCE (LIS)

C Program • Algorithm • Sample Output • Complexity Analysis

1. Aim

To write a C program to find the length and one actual longest increasing subsequence of a given sequence of integers, and to analyze its time and space complexity.

2. Problem Definition

Given an array of **n** integers, find a subsequence whose elements are in strictly increasing order and whose length is as large as possible. A subsequence is formed by deleting zero or more elements without changing the order of the remaining elements. The selected elements do not need to be adjacent.

Example: For the array 10, 22, 9, 33, 21, 50, 41, 60, one LIS is 10, 22, 33, 50, 60. Its length is 5.

3. Algorithm (Dynamic Programming)

1. Read the number of elements n and the n array elements.

2. If n is 0, report that the LIS length is 0.

3. Initialize dp[i] = 1 for every index i. Here, dp[i] stores the length of the LIS that ends at index i.

4. Initialize parent[i] = -1. This array stores the previous index in the selected subsequence.

5. For each i from 1 to n - 1, check every earlier index j from 0 to i - 1.

6. If a[j] < a[i] and dp[j] + 1 > dp[i], update dp[i] = dp[j] + 1 and parent[i] = j.

7. Track the index where dp[i] is maximum. This gives the end of one LIS.

8. Follow parent[] from that index to reconstruct the LIS, then print it in reverse order.

4. C Program

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n;

    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1 || n < 0) {
        printf("Invalid input.\\n");
        return 1;
    }

    if (n == 0) {
        printf("Length of LIS: 0\\n");
        printf("LIS: (empty)\\n");
        return 0;
    }

    int *a = (int *)malloc(n * sizeof(int));
    int *dp = (int *)malloc(n * sizeof(int));
    int *parent = (int *)malloc(n * sizeof(int));

    if (a == NULL || dp == NULL || parent == NULL) {
        printf("Memory allocation failed.\\n");
        free(a);
        free(dp);
        free(parent);
        return 1;
    }

    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &a[i]) != 1) {
            printf("Invalid input.\\n");
```

```
            free(a);
            free(dp);
            free(parent);
            return 1;
        }
    }

    int bestLength = 1;
    int bestEnd = 0;

    for (int i = 0; i < n; i++) {
        dp[i] = 1;
        parent[i] = -1;

        for (int j = 0; j < i; j++) {
            if (a[j] < a[i] && dp[j] + 1 > dp[i]) {
                dp[i] = dp[j] + 1;
                parent[i] = j;
            }
        }

        if (dp[i] > bestLength) {
            bestLength = dp[i];
            bestEnd = i;
        }
    }

    /* Reconstruct the LIS by following parent[] backwards. */
    int *lis = (int *)malloc(bestLength * sizeof(int));
    if (lis == NULL) {
        printf("Memory allocation failed.\\n");
        free(a);
        free(dp);
        free(parent);
        return 1;
    }

    int k = bestLength;
    int current = bestEnd;
    while (current != -1) {
        lis[--k] = a[current];
        current = parent[current];
    }

    printf("Length of LIS: %d\\n", bestLength);
    printf("One LIS: ");
    for (int i = 0; i < bestLength; i++) {
        printf("%d", lis[i]);
        if (i < bestLength - 1) {
            printf(" ");
        }
    }
    printf("\\n");

    free(a);
    free(dp);
    free(parent);
    free(lis);

    return 0;
}
```

5. Sample Input and Output

```
Enter number of elements: 8
Enter 8 elements: 10 22 9 33 21 50 41 60
Length of LIS: 5
One LIS: 10 22 33 50 60
```

The program may produce a different valid LIS when more than one longest increasing subsequence exists. The length will still be correct.

6. Time Complexity Analysis

Operation	Work performed	Time
Initialization	Initialize dp[] and parent[]	O(n)
Nested loops	For each i, examine all j < i	$O(n^2)$
Reconstruction	Follow parent[] for at most n elements	O(n)
Total	Dominated by the nested loops	$O(n^2)$

Why $O(n^2)$? The outer loop runs n times. For each i, the inner loop checks i earlier elements. The total number of comparisons is $0 + 1 + 2 + ... + (n - 1) = n(n - 1)/2$, which is $O(n^2)$.

Space Complexity: O(n). The input array, dp array, parent array, and LIS reconstruction array each use linear space. Therefore, the total auxiliary/storage space is O(n).

7. Result

The C program successfully finds the length of the Longest Increasing Subsequence and prints one such subsequence. Using the dynamic programming approach, the time complexity is **$O(n^2)$** and the space complexity is **O(n)**.